

0.68.3. Voir la position du A en direct — en simple

Le 0.68 en simple : `deplace_croix`, puis les deux nombre. Trois lignes dans la boucle.

1. Ce qu'il faut comprendre

C'est le 0.68, en plus simple : la croix tient en une ligne, `deplace_croix` ; il reste les deux `nombre` qui montrent la position.

Trois lignes font tout : bouger, montrer la colonne, montrer la ligne.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1234 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      73 o ← la console, à l'allumage – la mise en route de la console
deplace_croix() 217 o ← main() – une fonction fournie par le langage
main()          52 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre – attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), poser() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
EcrireNombre    33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
LireManette     37 o ← bouton() – lit les huit boutons
Tuiles         704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
bouton()      × 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
nombre()      × 2    20 o sur place, dans main() + EcrireNombre (33 o)
effacer()     × 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
poser()       × 1    13 o sur place, dans deplace_croix() + AdresseCase (10 o)
image()       × 1     3 o sur place, dans main() + AttendreImage (28 o)
images()      × 1     2 o sur place, dans deplace_croix()
```

Tes fonctions appelées :

```
deplace_croix() × 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x et y : voir l'onglet variables.h

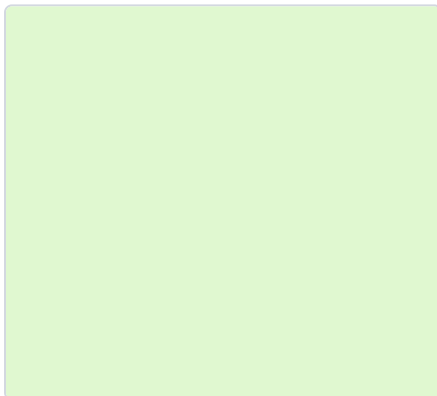
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>         // écrit un nombre en chiffres
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

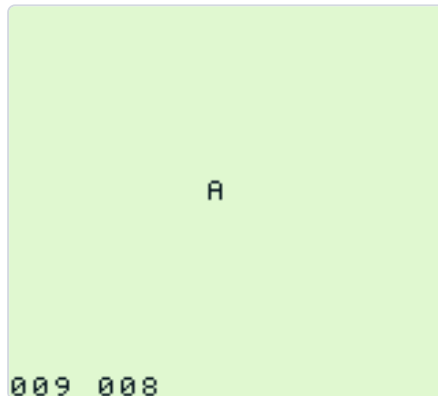
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133); // bouger
        nombre(0, 17, x);                      // la colonne, en direct
        nombre(4, 17, y);                      // la ligne, en direct
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.68 : le A bouge, sa position s'affiche en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ l'écran montre la colonne

✓ et la ligne

